C++ PROGRAMMING

Installation and Setup Development Environment for C++

To write and run C++ programs, we need a **development environment**. A development environment usually includes:

1. **Text Editor / IDE** – used to write code
2. **Compiler** – converts C++ code into machine language
3. **Debugger** – helps find errors in programs

What is a Compiler?

A **compiler** is a program that converts the human-readable C++ code into **machine code (binary)** that the computer can understand.
For beginners on Windows, the most common setup is:

- **MinGW (compiler)**
- **VS Code (editor)**

Installing MinGW (Compiler)

MinGW stands for **Minimalist GNU for Windows**.

1. Go to **SourceForge website**
2. Download **MinGW installer**
3. Run the installer
4. Select packages such as:
   - mingw32-gcc
   - mingw32-g++
   - mingw32-gdb
   - mingw32-make

These are required for compiling and debugging C++ programs.

5. Install the packages.

Setting Environment Variable (PATH)

After installing MinGW:

1. Go to **System Properties**
2. Click **Environment Variables**
3. Select **Path**
4. Add the path: → C:\MinGW\bin

This allows the system to run compiler commands like:
g++ --version
gcc --version
from the terminal.

Installing VS Code

Steps:

1. Download **VS Code**
2. Install it
3. Open VS Code
4. Install extensions:

Required extension:

- **C/C++ Extension by Microsoft**

This extension provides:

- Syntax highlighting
- Code completion
- Debugging support

Running First Program

Steps in terminal:

1. To compile program

```
g++ program.cpp
```

2. To run program

```
a.exe
```

The Need of C++

C++ was developed to overcome the **limitations of the C programming language**.
C is a **procedural language**, meaning it focuses on functions and procedures. However, as software systems became larger and more complex, C was not sufficient.

Problems with C

1. Difficult to manage **large software projects**
2. No support for **Object-Oriented Programming**
3. Limited **data security**
4. Poor **code reuse**
5. Difficult **maintenance**

Why C++ Was Needed

C++ introduced **Object-Oriented Programming (OOP)** concepts which help in building large and complex systems. C++ provides:

1. Data Abstraction
Only necessary information is shown to the user.

2. Encapsulation
Data and functions are combined into a single unit called a **class**.

3. Code Reusability
Through **inheritance**, one class can reuse the properties of another class.

4. Polymorphism
Allow functions to behave differently based on context.

5. Modularity
Programs are divided into smaller components (classes).

6. Better Security
Data hiding protects sensitive information.

Applications of C++

C++ is widely used in:

- Game development
- Operating systems
- Embedded systems
- Database systems
- High-performance applications
- Compilers

Features of C++

C++ has many powerful features.

1. Object-Oriented Programming

C++ supports OOP concepts:

- Class
- Object
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

2. Simple and Efficient

C++ is an extension of C, so it is easy for C programmers to learn.

3. Fast Execution

C++ programs are compiled into machine code, so they execute very fast.

4. Platform Independent (Portable)

Programs written in C++ can run on different platforms with minimal changes.

5. Rich Standard Library

C++ provides many built-in libraries like:

- iostream
- string
- vector
- algorithm

These help programmers develop programs quickly.

6. Memory Management

C++ allows dynamic memory allocation using:

new

delete

Example:

```
int *p = new int;
delete p;
```

7. Function Overloading
Multiple functions can have the same name but different parameters.
Example:

```
int sum(int a,int b);
double sum(double a,double b);
```

8. Operator Overloading
Operators like +, -, * can be customized for user-defined objects.

9. Exception Handling
Used to handle runtime errors.
Keywords:

- try
- catch
- throw

C++ Versus C

Feature	C	C++
Programming Type	Procedural	Object-Oriented
Focus	Functions	Objects
Data Security	Less	More
Code Reusability	Limited	High
Classes	Not available	Available
Inheritance	Not supported	Supported
Polymorphism	Not supported	Supported
Function Overloading	Not supported	Supported
Operator Overloading	Not supported	Supported
Exception Handling	Not supported	Supported

C Program

```
#include<stdio.h>

int main()
{
   printf("Hello");
   return 0;
}
```

C++ Program

```
#include<iostream>
using namespace std;
int main()
{
   cout<<"Hello";
   return 0;
}
```

History of C++

C++ was developed by **Bjarne Stroustrup** in **1979** at **Bell Labs**.

Timeline of C++ Development :

1979
Bjarne Stroustrup started developing a new language called:
C with Classes
It added object-oriented features to C.

1983
The language was renamed **C++**.
++ means **increment operator in C**, indicating improvement over C.

1985
First version of C++ was released.
The first C++ book was published:
The C++ Programming Language

Later Developments
Different versions of C++ were released:

Version	Year
C++98	1998
C++03	2003
C++11	2011
C++14	2014
C++17	2017
C++20	2020

Each version added new features to improve the language.

Writing Your First C++ Program

The traditional first program in any programming language prints:
Hello World

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   cout<<"Hello World";
   return 0;
}
```

Explanation of the Program

1. #include<iostream>
This line includes the **input-output stream library**.
It allows us to use:

- cout (output)
- cin (input)

2. using namespace std;
This allows us to use standard library names without writing std::.
Example without namespace:
std::cout
Example with namespace:
cout

3. int main()
The **main function** is the starting point of every C++ program.
Execution of the program begins from main().

4. { }
Curly braces define the body of the function.

5. cout
cout stands for **character output**.
It prints output on the screen.
Example:
cout<<"Hello World";

6. return 0;
This statement ends the program and returns control to the operating system.
0 means successful execution.

Steps of Program Execution

1. Write the program
2. Compile the program
3. Fix errors if any
4. Run the program
5. View the output

Example Execution

Compile
g++ hello.cpp
Run
a.exe
Output
Hello World

C++ Program Structure

A **C++ program structure** defines how a program is organized and written. Every C++ program follows a specific structure that makes it easier to read, understand, and maintain.
A typical C++ program consists of the following parts:

1. Documentation Section
2. Link Section
3. Namespace Declaration
4. Global Declaration Section
5. Class Definition
6. Main Function
7. Function Definitions

Example of a Basic C++ Program

```
// Documentation Section
#include<iostream>

// Namespace Declaration
using namespace std;

// Global Declaration
int x = 10;

// Class Definition
class Demo
{
   int a;
   public:
   void display()
   {
      cout<<"Value of a";
   }
};

// Main Function
int main()
{
   Demo d;
   d.display();
   return 0;
}
```

Explanation of Each Section

1. Documentation Section

This section contains **comments** that describe the program.
Example:

```
// This program displays Hello World
```

Comments are ignored by the compiler.

Types of comments:
Single-line comment
// comment
Multi-line comment
/* comment */

2. Link Section
This section includes **header files** required for the program.
Example
#include<iostream>
#include<math.h>
Header files provide built-in functions and libraries.
Example:

Header File	Purpose
iostream	Input and output
cmath	Mathematical functions
string	String handling

3. Namespace Declaration
Example
using namespace std;
The **std namespace** contains standard library components like:

- cout
- cin
- endl
- string

Without namespace:
std::cout
With namespace:
cout

4. Global Declaration Section
Variables declared outside functions are called **global variables**.
Example
int x = 10;
These variables can be accessed by all functions.

5. Class Definition
A **class** is a user-defined data type that contains:

- data members
- member functions

Example

```
class Student
{
   int roll;
   void display();
};
```

6. Main Function
The **main() function** is the entry point of a C++ program.
Example
```
int main()
{
   cout<<"Hello";
}
```
Program execution always starts from main().

7. Function Definitions
Functions perform specific tasks.
Example
```
void display()
{
   cout<<"Welcome";
}
```

Functions improve:
- readability
- modularity
- code reuse

Introduction to Advanced C++ Concepts and Features of C++17

Modern C++ introduced many advanced programming features to make programs **faster, safer, and easier to write**.
C++17 is an improved version of the C++ standard released in **2017**.

Some Important Advanced Concepts in C++

1. Object-Oriented Programming
C++ supports OOP concepts such as:
- Classes
- Objects
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction

```
class Car
{
public:
   void start()
   {
      cout<<"Car Started";
   }
};
```

2. Templates
Templates allow writing **generic programs**.
This function works with different data types.

```
template <class T>
T add(T a, T b)
{
   return a + b;
}
```

3. Exception Handling
Exception handling is used to handle runtime errors.
Keywords used:

- try
- catch
- throw

```
try
{
   if(x==0)
   throw x;
}
catch(int x)
{
   cout<<"Error";
}
```

4. Standard Template Library (STL)
STL provides powerful containers and algorithms.
Examples of containers:

- vector
- list
- stack
- queue
- map

Features of C++17

1. Structured Bindings
Allows unpacking values from structures.
Example

```
auto [x, y] = pair<int,int>(10,20);
```

2. if with Initializer
Example

```
if(int x = 10; x > 5)
{
   cout<<"Greater";
}
```

3. Inline Variables
Variables can now be declared **inline** in header files.
Example

```
inline int x = 10;
```

4. std::optional
Used to represent values that may or may not exist.
Example

```
optional<int> value;
```

5. Filesystem Library
Allows working with files and directories.
Example

```
#include <filesystem>
```

C++ Tokens

Tokens are the smallest units of a C++ program.
The compiler breaks the program into tokens during compilation.

Types of Tokens in C++

1. Keywords
2. Identifiers
3. Constants
4. Strings
5. Operators
6. Special Symbols

1. Keywords

Keywords are **reserved words** with special meanings.

Keyword

int

float

if

while

return

class

public

private, etc.

2. Identifiers

Identifiers are **names given to variables, functions, classes, etc.**
Example
age
studentName
totalMarks
Example program
int marks;

Rules for Identifiers

1. Must start with a letter or underscore
2. Cannot start with a number
3. Cannot be a keyword
4. No spaces allowed

3. Constants

Constants are **fixed values that do not change during program execution.**
Example :
10 → Integer constant
3.14 → Floating-point constant
'A' → Character constant
"Hello" → String constant

```
const int x = 10;        // Here 10 is a constant and x is a variable
```

4. Strings
Strings are sequences of characters enclosed in **double quotes**.
Example :
"Hello World"

5. Operators
Operators perform operations on variables.
+, -, *, /, %
Example :
int sum = a + b;

6. Special Symbols
Symbols used in programs.
Examples :
{ } ( ) ; , #

Initialization

Initialization means assigning a value to a variable at the time of declaration.
Example
int a = 10;
Here:

- variable → a
- value → 10

Types of Initialization

1. Direct Initialization
int x = 5;

2. Constructor Initialization
int x(10);

3. Uniform Initialization (C++11)
int x{10};
This method prevents type conversion errors.

Type	Syntax	Explanation
Direct Initialization	int x = 5;	Assigns value using = at declaration
Constructor Initialization	int x(5);	Value passed in parentheses
Uniform Initialization	int x{5};	Uses {} and prevents type conversion
Default Initialization	int x;	No value assigned (garbage for local)
Zero Initialization	int x{};	Automatically initializes to 0
Constant Initialization	const int x = 10;	Value cannot be changed

---

Static Members

In C++, **static members belong to the class rather than objects**.
Only **one copy** of a static variable exists for the entire class.

Static Data Member

Example

```
class Test
{
   static int count;
};
```

Characteristics

1. Shared by all objects
2. Stored only once in memory
3. Declared inside class but defined outside

Example

```
#include<iostream>
using namespace std;

class Test
{
public:
   static int count;

   Test()
   {
      count++;
   }
};

int Test::count = 0;

int main()
{
   Test t1,t2,t3;

   cout<<Test::count;

   return 0;
}
```

Output

3

Real-World Example:

Consider a ride booking application used for daily transportation services.

Each ride created in the system contains its own specific details such as:

- *Ride ID (unique for every ride)*
- *Distance travelled by the customer*

*Apart from these individual details, the application also defines a **fixed price per kilometer**, which is decided by the company based on policies, fuel cost, and market conditions.*

This price per kilometer:

- *remains the same for all rides at a particular time*
- *is not dependent on any single ride*
- *is centrally controlled by the system*

Static Member Function

Static functions access only **static data members**.

Example

```
class Demo
{
   static int x;

public:
   static void show()
   {
      cout<<x;
   }
};
```

In a login system:

- *Each user has:*
  - *Username*
  - *Password*
- *The system defines:*
  - *Common password rules (minimum length, special characters, etc.)*

A function that:

- *validates whether a password meets security rules*

*does not depend on a specific user object but checks against **common validation rules**.*
*Hence, it is best implemented as a **static member function**.*

Constant Members

Constant members are **members whose values cannot be modified**.
C++ provides:

1. Constant variables
2. Constant member functions

Constant variables

Value cannot be changed later.

Example :

```
const int x = 10;
```

In a login/verification system:

- *Each OTP is generated dynamically*
- *But the **expiry time (e.g., 5 minutes)** is fixed*

*Hence, **OTP expiry duration** is a **constant variable**.*

Constant Member Function

These functions **do not modify object data**.

Example :

```
class Demo
{
   int x;

public:
   int getValue() const
   {
      return x;
   }
};
```

In a GPS navigation system:

- *The system maintains:*
  - *Current latitude*
  - *Current longitude*

A function is used to:

- *fetch and display the current location*

This function:

- *only retrieves stored data*
- *does not change the location values*

*Hence, it is best implemented as a **constant member function**.*

const ensures that the function does not modify data members.

Expressions in C++

An **expression** is a combination of:

- variables
- constants
- operators
- function calls

that produces a value.

Example
a + b
Example program
int c = a + b;
Here
a + b
is the expression.

Operators in C++

An **operator** is a symbol used to perform operations on variables and values.
Operators act on **operands** to produce a result.

Types of Operators in C++

C++ provides many types of operators. Important operators include:

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.
These operators work with numeric values such as integers and floating-point numbers.

Operator	Meaning	Explanation	Example
+	Addition	Adds two values	10 + 5 = 15
-	Subtraction	Subtracts second from first	10 - 5 = 5
*	Multiplication	Multiplies two values	10 * 5 = 50
/	Division	Divides one value by another	10 / 5 = 2
%	Modulus	Gives remainder of division	10 % 5 = 0

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10, b = 5;
```

```
    cout<<"Addition = "<<a + b<<endl;          // Addition = 15
    cout<<"Subtraction = "<<a - b<<endl;       // Subtraction = 5
    cout<<"Multiplication = "<<a * b<<endl;    // Multiplication = 50
    cout<<"Division = "<<a / b<<endl;          // Division = 2
    cout<<"Modulus = "<<a % b<<endl;           // Modulus = 0

    return 0;
}
```

2. Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Meaning	Explanation	Example
=	Assign	Assigns value to variable	a = 5
+=	Add and assign	Adds and stores result	a=5; a+=3 → 8
-=	Subtract and assign	Subtracts and stores result	a=5; a-=2 → 3
*=	Multiply and assign	Multiplies and stores result	a=5; a*=2 → 10
/=	Divide and assign	Divides and stores result	a=6; a/=2 → 3
%=	Modulus and assign	Stores remainder	a=5; a%=2 → 1

Example Program

```
#include<iostream>
using namespace std;

int main()
{
    int a = 10;

    a += 5;
    cout<<"a += 5 → "<<a<<endl;       // a += 5 → 15
    a -= 3;
    cout<<"a -= 3 → "<<a<<endl;       // a -= 3 → 12
    a *= 2;
    cout<<"a *= 2 → "<<a<<endl;       // a *= 2 → 24
    a /= 4;
    cout<<"a /= 4 → "<<a<<endl;       // a /= 4 → 6
    return 0;
}
```

3. Relational Operators

Comparison operators are used to compare two values and return true or false.

Operator	Meaning	Explanation	Example
==	Equal to	Checks if values are equal	5 == 5 → true
!=	Not equal to	Checks if values are different	5 != 3 → true
>	Greater than	Checks if left is greater	5 > 3 → true
<	Less than	Checks if left is smaller	3 < 5 → true
>=	Greater than or equal	Checks if left ≥ right	5 >= 5 → true
<=	Less than or equal	Checks if left ≤ right	3 <= 5 → true

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10, b = 5;

   cout<<(a > b)<<endl;      //1
   cout<<(a < b)<<endl;      //0
   cout<<(a == b)<<endl;     //0

   return 0;
}
```

4. Logical Operators

Logical operators are used to combine multiple conditions.

Operator	Meaning	Explanation	Example
&&	Logical AND	True if both conditions are true	(5>3 && 3>1) → true
||	Logical OR	True if at least one is true	(5>3 || 3<1) → true
!	Logical NOT	Reverses the condition	!(5>3) → false

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10, b = 5;

   cout<<((a > 5) && (b > 2))<<endl; //1
   cout<<((a > 5) || (b < 2))<<endl;     //1
   cout<<!(a > b)<<endl;                 //0

   return 0;
}
```

5. Bitwise Operators

Bitwise operators work on binary representation of numbers.

Operator	Meaning	Explanation	Example
&	Bitwise AND	Sets bit if both bits are 1	5 & 3 → 1
|	Bitwise OR	Sets bit if at least one bit is 1	5 | 3 → 7
^	Bitwise XOR	Sets bit if bits are different	5 ^ 3 → 6
~	Bitwise NOT	Inverts all bits	~5 → -6
<<	Left shift	Shifts bits left	5 << 1 → 10
>>	Right shift	Shifts bits right	5 >> 1 → 2

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 5, b = 3;

   cout<<(a & b)<<endl;      //1
   cout<<(a | b)<<endl;      //7
   cout<<(a ^ b)<<endl;      //6

   return 0;
}
```

6. Unary Operators

Unary operators work on a single operand.

Operator	Meaning	Explanation	Example
+	Unary plus	Indicates positive value	+a
-	Unary minus	Negates the value	-a
++	Increment	Increases value by 1	a=5; ++a → 6
--	Decrement	Decreases value by 1	a=5; --a → 4
!	Logical NOT	Reverses boolean value	!(5>3) → false

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 5;
   cout<<++a<<endl;          //6
   cout<<--a<<endl;          //5
   cout<<-a<<endl;           //-5

   return 0;
}
```

7. Ternary Operator

Used as a short form of if-else.

Operator	Meaning	Explanation	Example
?:	Ternary operator	Returns value based on condition	(5>3)?10:20 → 10

Example Program

```
#include<iostream>
using namespace std;
int main()
{
   int a = 10, b = 20;
   int max = (a > b) ? a : b;

   cout<<"Maximum = "<<max;        // Maximum = 20

   return 0;
}
```

8. Special Operators

Used for specific purposes like memory access and size.

Operator	Meaning	Explanation	Example
sizeof	Size of variable	Returns memory size	sizeof(int) → 4
&	Address-of	Gives memory address	&a
*	Pointer	Access value at address	*ptr
.	Dot operator	Access object members	obj.x
->	Arrow operator	Access members via pointer	ptr->x

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10;
   int *ptr = &a;

   cout<<"Size = "<<sizeof(a)<<endl;              // Size = 4
   cout<<"Address = "<<&a<<endl;                  // Address = (memory address)
   cout<<"Value using pointer = "<<*ptr<<endl;            // Value using pointer = 10
   return 0;
}
```

Example Showing Multiple Operators

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10, b = 5;

   cout<<"Arithmetic "<<a+b<<endl;
   cout<<"Relational "<<(a>b)<<endl;
   cout<<"Logical "<<(a>5 && b<10)<<endl;

   return 0;
}
```

Control Statements and Arrays in C++

Control statements are used to **control the flow of execution of a program**.
Normally, programs execute statements **sequentially**, but control statements allow programs to make **decisions, repeat tasks, and jump to different parts of the program**.
Control statements are mainly divided into:

1. Decision-making statements
2. Looping statements
3. Jump statements

Decision Making Statements

Decision-making statements allow the program to **choose between different options based on a condition**.
The main decision-making statements in C++ are:

- if
- if-else
- else-if ladder
- switch

If Statement

The **if statement** is used to execute a block of code **only when a condition is true**.
Syntax

```
if(condition)
{
  statements;
}
```

If the condition is **true**, the statements inside the block are executed.
If the condition is **false**, the block is skipped.
Example

```
#include<iostream>
using namespace std;

int main()
{
   int age = 18;

   if(age >= 18)
   {
      cout<<"Eligible for voting";
   }

   return 0;
}
```

If-Else Statement

The **if-else statement** is used when we want to execute **one block if the condition is true and another block if the condition is false**.

Syntax

```
if(condition)
{
  statements1;
}
else
{
  statements2;
}
```

Example

```
#include<iostream>
using namespace std;

int main()
{
   int number;

   cout<<"Enter a number:";
   cin>>number;
   if(number % 2 == 0)
      cout<<"Even number";
   else
      cout<<"Odd number";
   return 0;
}
```

Else-If Ladder

When multiple conditions need to be checked, we use the **else-if ladder**.

Syntax

```
if(condition1)
{
  statements1;
}
else if(condition2)
{
  statements2;
}
else if(condition3)
{
  statements3;
}
else
{
  statements4;
}
```

The program checks each condition **one by one**.

Example

```
#include<iostream>
using namespace std;

int main()
{
   int marks;

   cout<<"Enter marks:";
   cin>>marks;

   if(marks >= 75)
      cout<<"Distinction";
   else if(marks >= 60)
      cout<<"First Class";
   else if(marks >= 50)
      cout<<"Second Class";
   else
      cout<<"Fail";

   return 0;
}
```

Switch Statement

The **switch statement** is used to execute one block of code from multiple options.
It is often used as an alternative to **multiple if-else statements**.

Syntax

```
switch(expression)
{
  case value1:
     statements;
     break;

  case value2:
     statements;
     break;

  default:
     statements;
}
```

Important Points

1. break is used to terminate the case.
2. default executes if none of the cases match.

When a user places an order, they are provided with multiple payment options such as:

- *Credit Card*
- *Debit Card*
- *UPI*
- *Cash on Delivery*

*Each payment method requires a **different processing flow**:*

- *Credit/Debit Card → requires card details and OTP verification*
- *UPI → requires UPI ID and app confirmation*
- *Cash on Delivery → confirms order without online transaction*

*The system takes the user's choice as input and must execute the **corresponding payment process**.*
*Since there are **multiple predefined options** and only **one option is selected at a time**, the `switch` statement is used to:*

- *match the selected option*
- *execute the appropriate block of code for that payment method*

Example

```
#include<iostream>
using namespace std;

int main()
{
   int choice;
   cout<<"1.Add\n2.Subtract\n";
   cin>>choice;

   switch(choice)
   {
      case 1:
         cout<<"Addition selected";
         break;

      case 2:
         cout<<"Subtraction selected";
         break;

      default:
         cout<<"Invalid choice";
   }

   return 0;
}
```

Looping Statements

Loops are used to **execute a block of code repeatedly until a condition becomes false**.
Types of loops:

1. for loop
2. while loop
3. do-while loop

For Loop

The **for loop** is used when the **number of iterations is known**.

Syntax

```
for(initialization; condition; increment/decrement)
{
   statements;
}
```

Parts of for loop

1. Initialization → starting value
2. Condition → loop continues while true
3. Increment/Decrement → updates variable

Example

```
#include<iostream>
using namespace std;

int main()
{
   for(int i=1;i<=5;i++)
   {
      cout<<i<<" ";
   }

   return 0;
}
```

Output

1 2 3 4 5

While Loop

The **while loop** executes the block **as long as the condition is true**.
It is called an **entry-controlled loop** because the condition is checked **before execution**.

Syntax

```
while(condition)
{
  statements;
}
```

Example

```
#include<iostream>
using namespace std;

int main()
{
   int i = 1;

   while(i <= 5)
   {
      cout<<i<<" ";
      i++;
   }

   return 0;
}
```

Do-While Loop

The **do-while loop** executes the block **at least once**.
This is called an **exit-controlled loop**.

Syntax

```
do
{
  statements;
}
while(condition);
```

Example

```
#include<iostream>
using namespace std;

int main()
{
   int i = 1;

   do
   {
      cout<<i<<" ";
      i++;
   }
   while(i <= 5);

   return 0;
}
```

Difference Between Loops

Loop	Condition Checked	Execution
for	Beginning	Known iterations
while	Beginning	Unknown iterations
do-while	End	Executes at least once

Jump Statements

Jump statements transfer control from one part of the program to another.
Types:

- break
- continue
- return

break Statement

The **break statement** terminates the loop or switch statement immediately.

Example

```
#include<iostream>
using namespace std;

int main()
{
   for(int i=1;i<=10;i++)
   {
      if(i==5)
         break;

      cout<<i<<" ";
   }
   return 0;
}
```

Output
1 2 3 4

continue Statement

The **continue statement** skips the current iteration and moves to the next iteration.

Example

```
#include<iostream>
using namespace std;

int main()
{
   for(int i=1;i<=5;i++)
   {
      if(i==3)
         continue;
         cout<<i<<" ";
   }

   return 0;
}
```

Output
1 2 4 5

return Statement

The **return statement** terminates a function and returns a value.

Example

```
int sum(int a,int b)
{
   return a+b;
}
```

Arrays in C++

An **array** is a collection of elements of the same data type stored in **contiguous memory locations**.

Example

Instead of writing:

marks1
marks2
marks3
marks4

We can use an array:

marks[4]

In a cricket match, especially in formats like a T20 match:

- *The game consists of a fixed number of overs (e.g., **20 overs**)*
- *In each over, the batting team scores a certain number of runs*

So, instead of storing total runs directly, the system records:

- ***runs scored in each over separately***

For example:

- *Over 1 → 8 runs*
- *Over 2 → 12 runs*
- *Over 3 → 6 runs*
- *… up to 20 overs*

Using an array makes it easy to:

- *calculate **total score** by adding all elements*
- *find the **highest scoring over***
- *analyze performance (e.g., slow start, strong finish)*
- *access or update runs of any specific over quickly*

Declaration of an Array

Array declaration syntax:

data_type array_name[size];

Example

```
int arr[5];
```

This creates an array that can store **5 integers**.

Initialization of an Array

Array values can be initialized during declaration.

Example:

```
int arr[5] = {10,20,30,40,50};
```

If size is not specified:

```
int arr[] = {1,2,3,4};
```

Compiler automatically determines the size.

Accessing Array Elements

Array elements are accessed using **index numbers**.

Index always starts from **0**.

Example

arr[0]
arr[1]
arr[2]

Example program

```
#include<iostream>
using namespace std;

int main()
{
   int arr[3] = {10,20,30};

   cout<<arr[0]<<endl;
   cout<<arr[1]<<endl;
   cout<<arr[2]<<endl;

   return 0;
}
```

1-D Array

A **1-D array** is a list of elements stored in a single row.
Example
10 20 30 40 50
Declaration

```
int arr[5];
```

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int arr[5] = {10,20,30,40,50};

   for(int i=0;i<5;i++)
   {
      cout<<arr[i]<<" ";
   }

   return 0;
}
```

2-D Array

A **2-D array** is an array of arrays and is used to represent **tables or matrices**.
Example
1 2 3
4 5 6
7 8 9
Declaration

```
data_type array_name[rows][columns];
```

Example

```
int arr[3][3];
```

Initialization of 2-D Array

Example

```
int arr[2][3] =
{
   {1,2,3},
   {4,5,6}
};
```

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int arr[2][2] = {{1,2},{3,4}};

   for(int i=0;i<2;i++)
   {
      for(int j=0;j<2;j++)
      {
         cout<<arr[i][j]<<" ";
      }
      cout<<endl;
   }

   return 0;
}
```

Output

```
1 2
3 4
```

In a cinema hall:

- *Seats are organized in a structured format of **rows and columns** (for example: Row A, Row B, Row C, and seat numbers like 1, 2, 3, etc.)*
- *Each seat has a **status**, such as:*
  - *Booked*
  - *Available*

To manage this efficiently, the system needs to store the status of every seat.

Instead of creating separate variables for each seat like:

- *seatA1, seatA2, seatA3, …*

*the system uses a **2D array** to represent the entire seating arrangement.*

Functions in C++

A **function** is a block of code that performs a specific task. Functions help in dividing a large program into smaller and manageable parts.

Functions improve:

- Code readability
- Code reuse
- Program maintenance
- Modularity

Instead of writing the same code many times, we can write a **function once and call it whenever needed**.

Basic Structure of a Function

A function in C++ has the following components:

1. Function declaration (prototype)
2. Function definition
3. Function call

Example Program

```
#include<iostream>
using namespace std;

void greet()   // function definition
{
   cout<<"Hello Student";
}

int main()
{
   greet();   // function call
   return 0;
}
```

Output

Hello Student

Explanation:

- void → return type
- greet() → function name
- {} → body of the function

In a food delivery application:

- *A user orders multiple food items*
- *The system needs to calculate the **total bill**, which includes:*
  - *Price of items*
  - *Taxes*
  - *Delivery charges*

*This calculation needs to be performed **every time an order is placed**.*

*Instead of writing the same calculation logic repeatedly, the system defines a **function** to compute the total bill.*

*A **function** is used to perform a **specific task**, such as calculating the total cost of an order.*

In this example:

- *The billing logic is written once inside a function*
- *It can be reused for every order*

General Syntax of a Function

```
return_type function_name(parameter_list)
{
   function body
}
```

Example

```
int add(int a,int b)
{
   return a+b;
}
```

Different Forms of Functions

Functions can be categorized based on:

- presence of arguments
- presence of return values

There are **four main forms of functions**.

1. Function with No Arguments and No Return Value

This type of function **does not accept parameters and does not return any value**.

Syntax

```
void function_name()
{
  statements;
}
```

Example

```
#include<iostream>
using namespace std;

void display()
{
   cout<<"Welcome to C++ Programming";
}

int main()
{
   display();
   return 0;
}
```

2. Function with Arguments but No Return Value

The function **accepts parameters but does not return any value**.

Syntax

```
void function_name(parameters)
{
  statements;
}
```

Example

```
#include<iostream>
using namespace std;

void add(int a,int b)
{
   cout<<"Sum = "<<a+b;
}

int main()
{
   add(10,20);
   return 0;
}
```

Output

```
Sum = 30
```

3. Function with Arguments and Return Value

The function **accepts parameters and returns a value**.

Syntax

```
return_type function_name(parameters)
{
  return value;
}
```

Example

```
#include<iostream>
using namespace std;

int add(int a,int b)
{
   return a+b;
}

int main()
{
   int result;

   result = add(10,20);

   cout<<"Sum = "<<result;

   return 0;
}
```

Output

```
Sum = 30
```

4. Function with No Arguments but Return Value

The function **does not take parameters but returns a value**.

Example

```
#include<iostream>
using namespace std;

int getNumber()
{
   int x = 10;
   return x;
}

int main()
{
   int num;

   num = getNumber();
```

```
    cout<<num;

    return 0;
}
```

Output
10

Function Prototyping

A **function prototype** is a declaration of a function before its actual definition.
It tells the compiler:

- function name
- return type
- parameters

Syntax

```
return_type function_name(parameters);
```

Example

```
#include<iostream>
using namespace std;

int add(int,int);   // function prototype

int main()
{
    int result;

    result = add(5,3);

    cout<<"Sum = "<<result;

    return 0;
}

int add(int a,int b)
{
    return a+b;
}
```

Advantages of Function Prototype

1. Allows functions to be defined **after main()**
2. Helps compiler check **parameter types**
3. Improves program organization

Call by Value

In **call by value**, a copy of the actual parameter is passed to the function.
Changes made inside the function **do not affect the original variable**.

Example

```
#include<iostream>
using namespace std;
void change(int x)
{
   x = 50;
}
int main()
{
   int a = 10;
   change(a);
   cout<<a;
   return 0;
}
```

Output
10
Explanation:
The original variable a remains unchanged.

In a photo editing application:

- *A user selects a photo and tries different filters (brightness, contrast, etc.)*
- *While previewing, the system applies changes to a **temporary copy of the image***

However:

- *The **original image remains unchanged** unless the user clicks "Save"*

In this case:

- *The function receives a **copy of the image data***
- *Any modifications (filter effects) are applied only to the copy*

Call by Reference

In **call by reference**, the address of the variable is passed to the function.
Changes made inside the function **affect the original variable**.
Reference is represented using **& operator**.

Example

```
#include<iostream>
using namespace std;
void change(int &x)
{
   x = 50;
}
int main()
{
   int a = 10;
   change(a);
   cout<<a;
   return 0;
}
```

Output
50
Explanation:
The original variable is modified.

In a shared document system:

- *Multiple users can edit the same document simultaneously*
- *When one user makes changes:*
  - *the **actual document is updated immediately***
  - *all users can see the changes in real time.*

In this case:

- *The function works on the **original document itself***
- *Any modification directly affects the main data*

Difference Between Call by Value and Call by Reference

Feature	Call by Value	Call by Reference
Parameter Passed	Copy of variable	Address of variable
Original Value	Not changed	Changed
Memory	Extra memory used	No extra memory
Speed	Slower	Faster

Inline Functions

An **inline function** is a function where the compiler replaces the function call with the actual function code.
This reduces the **function call overhead**.
Inline functions are declared using the **inline keyword**.

Syntax

```
inline return_type function_name(parameters)
{
  function body
}
```

Example

```
#include<iostream>
using namespace std;

inline int square(int x)
{
   return x*x;
}

int main()
{
   cout<<square(5);

   return 0;
}
```

Output
25

In a banking application:

- *Before every transaction, the system checks:*
  - *if balance is above minimum limit*

This check:

- *is simple*
- *is used repeatedly*

Using an inline function:

- *ensures faster validation*

Why Inline:
Real-time small updates

****
"Inline = small + repeated + fast execution."
****

Advantages of Inline Functions

1. Faster execution
2. Eliminates function call overhead
3. Improves performance for small functions

Disadvantages

Inline functions are not suitable for:
large functions
recursive functions

Math Library Functions

C++ provides many **mathematical functions** in the **cmath library**.
To use them, include the header file:→ #include<cmath>

Common Math Functions:

Function	Description
sqrt(x)	Square root
pow(x,y)	Power function
abs(x)	Absolute value
ceil(x)	Round up
floor(x)	Round down
sin(x)	Sine value
cos(x)	Cosine value
tan(x)	Tangent value
log(x)	Natural logarithm

Example Program

```
#include<iostream>
#include<cmath>
using namespace std;

int main()
{
   cout<<sqrt(25)<<endl;
   cout<<pow(2,3)<<endl;
   cout<<abs(-10)<<endl;

   return 0;
}
```

Output

```
5
8
10
```

Example Using Multiple Functions

```
#include<iostream>
#include<cmath>
using namespace std;

int square(int x)
{
```

```
    return x*x;
}

int main()
{
    int num = 5;
    cout<<"Square = "<<square(num)<<endl;
    cout<<"Square Root = "<<sqrt(num)<<endl;
    return 0;
}
```

Introduction to Memory Management in C++

What is Memory Management?

Memory management refers to the process of **allocating and deallocating memory during program execution**.

In programming, variables and objects require memory to store data. Efficient memory management ensures that memory is **used properly and released when it is no longer needed**.

C++ provides **two types of memory allocation**:

1. **Static Memory Allocation**
2. **Dynamic Memory Allocation**

1. Static Memory Allocation

Memory is allocated **at compile time**.

Examples:

- global variables
- static variables
- arrays with fixed size

Example:

```
int a = 10;
int arr[5];
```

Characteristics:

- Memory size is fixed.
- Cannot be changed during runtime.

In a classroom:

- *The number of students is **fixed** (e.g., 30 students)*
- *The teacher prepares:*
  - *30 answer sheets*
  - *30 seats*

This allocation:

- *is decided **before the class starts***
- *does not change during the session*

2. Dynamic Memory Allocation

Memory is allocated **during program execution (runtime)**.

In C++, dynamic memory allocation is done using:

- new
- delete

Example:

```
int *p = new int;
```

Advantages:

- Flexible memory usage
- Memory allocated as required

In an online webinar:

- *The number of participants is **not fixed***
- *People can register at any time*

The system:

- *allocates space for participants **as they register***
- *increases storage dynamically*
- *can also remove participants if they cancel*

Pointers in C++

What is a Pointer?

A **pointer** is a variable that stores the **memory address of another variable**.
Instead of storing the actual value, it stores the **location of the value in memory**.

Pointer Declaration

Syntax:
data_type *pointer_name;
Example:
int *p; // p is a pointer to an integer.

Example Program

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10;
   int *p;
   p = &a;
   cout<<"Value of a = "<<a<<endl;
   cout<<"Address of a = "<<&a<<endl;
   cout<<"Pointer value = "<<p<<endl;
   cout<<"Value using pointer = "<<*p<<endl;

   return 0;
}
```

Explanation:
- & → address-of operator
- * → dereference operator

Arrays Using Pointers

Arrays and pointers are closely related in C++.
The **name of an array represents the address of its first element**.
Example:
int arr[5] = {10,20,30,40,50};
Here:
arr = &arr[0]

Accessing Array Elements Using Pointers

Example:

```
#include<iostream>
using namespace std;

int main()
{
   int arr[5] = {10,20,30,40,50};
   int *p;

   p = arr;
```

```
   for(int i=0;i<5;i++)
   {
      cout<<*(p+i)<<" ";
   }

   return 0;
}
```

Output
10 20 30 40 50

Explanation:
*(p+i) = arr[i]

Enumeration (enum) in C++

Enumeration is a user-defined data type used to assign names to integer constants.
It improves readability and code clarity.

1. Default Values

- By default, values start from **0**
- Each next value increases by **1**

enum Week {Mon, Tue, Wed};

Internally:

- Mon = 0
- Tue = 1
- Wed = 2

2. Assigning Custom Values
enum Week {Mon=1, Tue, Wed=5, Thu};

Values become:

- Mon = 1
- Tue = 2
- Wed = 5
- Thu = 6

Values auto-increment from last assigned value

3. Using enum Variables
enum Week {Mon, Tue, Wed};

Week day;
day = Tue;

4. Printing enum Values
cout << day;

Output is **integer value**, not name

Imagine you order a pizza from a food delivery app.
*From the moment you place the order, your pizza begins a **step-by-step journey**, where it passes through several clearly defined stages:*

- ***Ordered*** *→ Your request has been successfully placed and received by the restaurant*
- ***Baking*** *→ The pizza is being prepared and baked in the kitchen*
- ***OutForDelivery*** *→ The pizza is packed and handed over to the delivery person, who is on the way to your location*
- ***AtYourDoor*** *→ The pizza has reached you and the order is completed*

*At any given moment, your order can be in **only one of these stages**, and the system must track this status accurately to inform both the user and the restaurant.*
These stages are:

- ***fixed and predefined*** *(no random states)*
- ***occur in a logical sequence***
- ***mutually exclusive*** *(only one state at a time)*

*Instead of using unclear numeric values like 0, 1, 2, 3, the system uses **meaningful names** for each stage.*

5. enum with switch

```
#include<iostream>
using namespace std;

enum Week {Mon, Tue, Wed};

int main()
{
   Week day = Tue;

   switch(day)
   {
      case Mon: cout<<"Monday"; break;
      case Tue: cout<<"Tuesday"; break;
      case Wed: cout<<"Wednesday"; break;
   }

   return 0;
}
```

Best real-life usage of enum

enum vs int

Feature	enum	int
Readability	High	Low
Meaningful	Yes	No
Type safety	Better	Less

Typedef in C++

typedef is a **keyword in C and C++ used to create a new name (alias) for an existing data type**.
It does **not create a new data type**, but only **provides another name for an existing data type**.
This helps:

- Improve **code readability**
- Simplify **complex data types**
- Make programs **easier to maintain**

Syntax of typedef

typedef existing_data_type new_name;
Where:

- **existing_data_type** → original data type
- **new_name** → new alias name

Example

typedef int Integer;

Here:
Integer = int

Example Program

```
#include<iostream>
using namespace std;
typedef int Integer;


int main()
{
   Integer a = 10;
   Integer b = 20;
   cout<<"Sum = "<<a+b;

   return 0;
}
```

Now we can write:
Integer a = 10;
instead of
int a = 10;

Output
Sum = 30

Why Use typedef?

typedef is mainly used to:

1. **Improve readability of code**
2. **Simplify complex declarations**
3. **Provide meaningful names for data types**
4. **Make programs portable**

Example Showing Better Readability
Without typedef
unsigned long int salary;
With typedef
typedef unsigned long int Salary;

Salary s1;
This makes the code easier to understand.

Typedef with Arrays

typedef can also be used with arrays.
Example
typedef int Marks[5];
Now we can declare arrays like this:
Marks student1;
Marks student2;

Example program

```
#include<iostream>
using namespace std;
```

```
typedef int Marks[5];

int main()
{
   Marks m = {10,20,30,40,50};
   for(int i=0;i<5;i++){
   cout<<m[i]<<" ";
   }
   return 0;
}
```

Output
10 20 30 40 50

Typedef with Pointers

typedef can simplify pointer declarations.
Example
Without typedef
int *ptr;
With typedef
typedef int* IntPointer;

IntPointer p1,p2;
Both p1 and p2 are pointers.

Using new Operator

The **new operator** is used to **allocate memory dynamically** during runtime.
Syntax
pointer = new data_type;

Example

```
#include<iostream>
using namespace std;

int main()
{
   int *p;
   p = new int;
   *p = 25;
   cout<<*p;

   return 0;
}
```

In a hospital:

- *Patients arrive at different times*
- *The number of patients is unpredictable*

The system needs to:

- *create new records dynamically for each patient*

Using ***new****, memory is allocated:*

- *when a new patient record is created*

Dynamic Array Using New

Example
int *arr = new int[5];
This allocates memory for **5 integers dynamically**.

Class

A **class** is a blueprint or template that defines the structure and behavior of objects. It specifies:

- **Data members (attributes)**variables that describe the properties of the object.
- **Member functions (methods)**: functions that define the behavior of the object.

Example in your notes:

```
class Demo
{
public:
   int x;          // data member

   void display()   // member function
   {
      cout<<"Value = "<<x;
   }
};
```

Explanation:

- Demo is a **class**.
- It defines **what an object of type Demo will have**:
  - An integer x
  - A function display() that prints the value of x
- Notice that **no actual memory is allocated yet**
  for x when you define the class.
  The class is only a **design** — like a blueprint for a house.

*A **class** is like a **blueprint or design plan** for a house. It defines:*

- ***Properties (data members):** the features a house will have, such as:*
  - *Number of rooms*
  - *Number of doors and windows*
  - *Flooring type*
- ***Behaviors (member functions):** the actions or functionalities of the house, such as:*
  - *OpenDoor()*
  - *CloseWindow()*
  - *PaintWall()*
- *The blueprint **does not create a real house**; it only specifies **what a house should have and what it can do**.*
- *From this single blueprint, you can construct **many houses**, each with the same structure but possibly different colors, sizes, or interiors.*

***Class** = blueprint of a house. It defines the design, layout, and functions, but it doesn't occupy land or become a real house until you build it.*

Object

An **object** is a real instance of a class. It occupies memory and has its own copy of the data members defined in the class.

Example from your notes:

```
int main()
{
   Demo obj;      // obj is an object of class Demo

   Demo *ptr;     // pointer to a Demo object

   ptr = &obj;    // ptr stores the address of obj

   ptr->x = 10;   // accessing data member through pointer

   ptr->display(); // accessing member function through pointer
}
```

Explanation:

- obj is an **object** of class Demo.
- It **allocates memory** for the data member x.
- You can access x and display() using:
  - **Dot operator:** obj.x or obj.display()
  - **Pointer operator:** ptr->x or ptr->display()
- ptr->x is equivalent to (*ptr).x.

*An **object** is a real, tangible instance created from a class (blueprint). It has **actual memory allocated** for its properties and can perform the actions defined in the class.*
Example in the house analogy:

- *You use the blueprint (class) to **build a specific house**.*
- *Each house you build is an **object** of the blueprint.*

For instance:

- *House1 → 3 rooms, blue walls, wooden doors*
- *House2 → 4 rooms, green walls, glass doors*

Even though both houses come from the same blueprint:

- *Each house **occupies its own space (memory)***
- *Each house can **perform actions independently**, like opening doors or painting walls*

***Analogy:** Object = the actual constructed house that you can live in, decorate, and use.*

this Pointer

The **this pointer** is a special pointer available inside class member functions.
It refers to the **current object of the class**.

Syntax

this->variable

Example

```
#include<iostream>
using namespace std;


class Test
{
   int x;

public:
   void setValue(int x)
   {
      this->x = x;
   }

   void display()
   {
      cout<<x;
   }
};

int main()
{
   Test t;

   t.setValue(10);
   t.display();

   return 0;
}
```

Explanation

this->x

refers to the **current object's variable**.

*Imagine multiple houses built from the same blueprint. Each house has its own properties like color and number of rooms. Now, when some operation is performed—like painting or updating details—it must apply to the **specific house currently being worked on**, not to all houses.*
*Here, **"this house"** becomes important. It clearly identifies **which exact house** is being modified at that moment.*
In programming terms:

- *Each house = object*
- *The action being performed (like setting color) = member function*
- *"this house" = `this` pointer*

*So, whenever a function runs for an object, `this` automatically refers to **that specific object**.*
Sometimes, there can be confusion when the same name is used in two places:

- *One is the property of the house (like its color)*
- *Another is a temporary value given as input*

Without clarity, it becomes unclear which one we are referring to.
`this` removes the confusion by clearly indicating:
"Use the property of this object."

Understanding with Multiple Objects

- *When working on House 1 → `this` refers to House 1*
- *When working on House 2 → `this` refers to House 2*

*Even if the same instruction is used, each house updates **its own data independently** because `this` always points to the*

Comparison of new with malloc, calloc, realloc

Feature	new	malloc	calloc	realloc
Language	C++	C	C	C
Memory Allocation	Runtime	Runtime	Runtime	Resize memory
Constructor Call	Yes	No	No	No
Type Safety	Yes	No	No	No
Return Type	Typed pointer	void pointer	void pointer	void pointer
Memory Initialization	No	No	Yes (zero)	Depends

Example Using malloc
int *p = (int*) malloc(sizeof(int));
Example using new
int *p = new int;

Memory Freeing Using Delete Operator

Memory allocated using **new must be freed using delete**.
Otherwise it causes **memory leak**.
Syntax
delete pointer;

Example

```
#include<iostream>
using namespace std;

int main()
{
   int *p = new int;

   *p = 20;

   cout<<*p;

   delete p;

   return 0;
}
```

Deleting Dynamic Arrays

If memory is allocated using:
int *arr = new int[5];
It must be deleted using:
delete[] arr;

Memory Leak

A **memory leak** occurs when allocated memory is not released.
Example
int *p = new int;
If we do not write:
delete p;
memory remains occupied.

Object-Oriented Programming (OOP) in C++

Object-Oriented Programming (OOP) is a **programming paradigm based on objects and classes**.
In OOP, programs are organized around **data and the functions that operate on that data**.
Unlike procedural programming (like C), which focuses on **functions**, OOP focuses on **objects**.
OOP helps in building **large, complex, and reusable software systems**.
C++ is one of the most popular programming languages that supports **Object-Oriented Programming**.

Basic Concepts of Object-Oriented Programming

The main concepts of OOP are:

1. Class
2. Object
3. Encapsulation
4. Abstraction
5. Inheritance
6. Polymorphism

These concepts help programmers write **modular, reusable, and secure programs**.

Class

A **class** is a user-defined data type that represents a group of objects having similar properties and behaviors.
A class contains:

- **Data members** (variables)
- **Member functions** (functions)

It acts as a **blueprint for creating objects**.
Syntax of Class

```
class ClassName
{
   data members;
   member functions;
};
```

Example

```
#include<iostream>
using namespace std;
class Student
{
public:
   int roll;
   string name;
   void display()
   {
      cout<<"Roll: "<<roll<<endl;
      cout<<"Name: "<<name<<endl;
   }
};
```

Here:

- Student is a class.
- roll and name are data members.
- display() is a member function.

Object

An **object** is an instance of a class.
Objects are used to **access the data members and member functions of a class**.

Example

```
Student s1;
```

Here:

- s1 is an object of class Student.

Program Example

```
#include<iostream>
using namespace std;

class Student
{
public:
   int roll;
   string name;

   void display()
   {
      cout<<"Roll: "<<roll<<endl;
      cout<<"Name: "<<name<<endl;
   }
};
int main()
{
   Student s1;
   s1.roll = 1;
   s1.name = "Rahul";

   s1.display();

   return 0;
}
```

Access Specifiers in C++

Access specifiers control **how class members can be accessed**.
There are three access specifiers:

1. Public
2. Private
3. Protected

1. Public

Members declared as **public** can be accessed from **anywhere in the program**.

Example
```
class Demo
{
public:
   int x;
};
```

2. Private

Members declared as **private** can only be accessed **inside the class**.

Example
```
class Demo
{
private:
   int x;
};
```
Private members cannot be accessed directly by objects.

3. Protected

Members declared as **protected** can be accessed:

- inside the class
- in derived classes (inheritance)

Example
```
class Demo
{
protected:
   int x;
};
```

*Imagine a house where different areas have **different levels of access**.*

Public – Open Area (Accessible to Everyone)
*Public members are like the **main gate or doorbell of a house**.*
- *Anyone from outside can access it*
- *No restriction*

House Analogy
- *Doorbell*
- *House address*
- *Front gate*

Anyone (guests, delivery person, neighbors) can use these.

Private – Personal Room (Highly Restricted)
*Private members are like your **bedroom or locker**.*
- *Only you can access it*
- *Outsiders are **not allowed***

House Analogy
- *Bedroom*
- *Personal cupboard*
- *Safe locker*

Guests cannot enter your bedroom without permission.

Protected – Family Area (Limited Access)
*Protected members are like a **family-only area** in the house.*
- *Not open to outsiders*
- *Accessible to family members*

House Analogy
- *Kitchen*
- *Living room (for family use)*
- *Dining area*

Guests cannot freely use it, but family members can use it.

Access Specifier Table

Specifier	Accessible Inside Class	Accessible Outside Class	Accessible in Derived Class
Public	Yes	Yes	Yes
Private	Yes	No	No
Protected	Yes	No	Yes

Function Overloading

Function overloading means **having multiple functions with the same name but different parameters**.
The compiler differentiates functions based on:

- number of parameters
- type of parameters

Example

```
#include<iostream>
using namespace std;

class Demo
{
public:
   int add(int a,int b)
   {
      return a+b;
   }

   int add(int a,int b,int c)
   {
      return a+b+c;
   }
};

int main()
{
   Demo d;

   cout<<d.add(2,3)<<endl;
   cout<<d.add(2,3,4);

   return 0;
}
```

Output

```
5
9
```

Imagine an online payment system where a user can pay using different methods:

- *Credit Card*
- *UPI*
- *Net Banking*

Now, the system provides a single option:

"Make Payment"

*Even though the button says **"Make Payment"**, the process changes based on input:*

1. *If you enter **card details** → payment is processed via Credit Card*
2. *If you enter **UPI ID** → payment is processed via UPI*
3. *If you choose **bank details** → payment is processed via Net Banking*

- *Function name → `makePayment()` (same name)*
- *Input changes → Card details / UPI ID / Bank info*
- *Behavior changes → Different payment processing logic*

*Same function name, but **different execution depending on input***

Why This is Useful

- *You don't need different names like:*
  - `payByCard()`
  - `payByUPI()`
  - `payByBank()`

  Instead, you use one common name: ***`makePayment()`***

Inheritance

Inheritance is the process by which **one class acquires the properties and behaviors of another class**.

The class that is inherited from is called:

- **Base class (parent class)**

The class that inherits is called:

- **Derived class (child class)**

Example

```
#include<iostream>
using namespace std;
class Animal
{
public:
   void eat()
   {
      cout<<"Animal eats food"<<endl;
   }
};
class Dog : public Animal
{
public:
   void bark()
   {
      cout<<"Dog barks"<<endl;
   }
};
int main()
{
   Dog d;
   d.eat();
   d.bark();

   return 0;
}
```

Output

```
Animal eats food
Dog barks
```

Types of Inheritance

1. Single inheritance
2. Multiple inheritance
3. Multilevel inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

In an airport, many different people work together, but they all belong to one common category:

Airport Staff

*Even though their jobs are different, they share some **basic identity and information**.*

Common Features (Parent)

- *Name*
- *Employee ID*
- *Duty timing*

Specialized Features (Child)

- ***Pilot***
  - *Flight control*
  - *Navigation*
- ***Cabin Crew***
  - *Passenger safety*
  - *Service*
- ***Ground Staff***
  - *Baggage handling*
  - *Check-in process*

*All are **staff members**, but roles differ.*

*Every person working at the airport is first recognized as **staff***

Then, based on their job:

- *Some become pilots*
- *Some become cabin crew*
- *Some become ground staff*

So:

- ***Same base identity (Staff)***
- ***Different specialized roles***

Polymorphism

Polymorphism means **one name having multiple forms**.
In C++, polymorphism allows **functions or operators to behave differently in different situations**.
Polymorphism is mainly of two types:

1. Compile-time polymorphism
2. Runtime polymorphism

Compile-Time Polymorphism

Occurs during compilation.
Examples:

- Function overloading
- Operator overloading

Runtime Polymorphism

Occurs during program execution.
Achieved using:

- **Function overriding**
- **Virtual functions**

You use a navigation app like Google Maps and enter:

"Navigate to office"

Different Behaviors

- ***Car Mode***
  - *Shows fastest road route*
  - *Avoids traffic*
- ***Walking Mode***
  - *Shows shortcuts, footpaths*
- ***Bike Mode***
  - *Suggests bike-friendly routes*

Same action:
"Navigate to office"

Different behavior:
Depends on travel mode

"Same destination, different routes."

Example (Function Overriding)

```
#include<iostream>
using namespace std;

class Base
{
public:
   void show()
   {
      cout<<"Base class function"<<endl;
   }
};

class Derived : public Base
{
public:
   void show()
   {
      cout<<"Derived class function"<<endl;
   }
};

int main()
{
   Derived d;
   d.show();

   return 0;
}
```

Difference Between Compile-Time and Run-Time Polymorphism

Feature	Compile-Time Polymorphism	Run-Time Polymorphism
When resolved	Compile time	Run time
Binding	Early binding	Late binding
Techniques	Function overloading, Operator overloading	Function overriding, Virtual functions
Performance	Faster	Slightly slower

Advantages of Polymorphism

1. Code reusability
2. Flexibility
3. Improves readability
4. Supports dynamic method resolution
5. Helps implement OOP design

Namespaces in C++

A **namespace** is used to organize code and prevent **name conflicts**.
When two libraries contain functions with the same name, namespaces help avoid confusion.

Example

```
#include<iostream>
using namespace std;

namespace First
{
   int value = 10;
}
namespace Second
{
   int value = 20;
}
int main()
{
   cout<<First::value<<endl;
   cout<<Second::value;
   return 0;
}
```

Output

10
20

Explanation:
First::value
Second::value
Here :: is called the **scope resolution operator**.

*In the film industry, sometimes **different movies can have the same title**, but they are released in different years.*
For example:

- *"Hero" released in 1983*
- *"Hero" released in 2015*

*Even though the name is the same, these are **completely different movies** with different actors, storylines, and direction.*
If someone simply says:
"Let's watch Hero"
It creates confusion:

- *Which movie are we talking about?*
- *The 1983 version or the 2015 version?*

Because:
*The name alone is **not enough to uniquely identify it***
Solution (Using Namespace Concept)
To remove confusion, we include additional context:
"1983::Hero"
"2015::Hero"
Now it becomes very clear:

- *Each movie is uniquely identified*
- *Even though the names are the same*

Using Namespace

Instead of writing namespace every time, we can use:
using namespace namespace_name;
Example
using namespace std;
This allows us to use:
cout
cin
endl
without writing std:: .

Advantages of Namespaces

1. Avoids name conflicts
2. Organizes large programs
3. Improves code readability
4. Useful in large projects and libraries

Constructors in C++

A **constructor** is a special member function of a class that is **automatically called when an object of the class is created**.
Constructors are mainly used to **initialize the data members of a class**.

Important Characteristics of Constructors

1. Constructor name must be **same as the class name**
2. It **does not have a return type** (not even void)
3. It is **automatically invoked** when an object is created
4. It is used for **initializing objects**

Example of Constructor

```
#include<iostream>
using namespace std;

class Student
{
public:
   int roll;
   Student()
   {
      roll = 10;
   }
};

int main()
{
   Student s1;

   cout<<s1.roll;
}
```

Output
10
Explanation:
When the object s1 is created, the constructor **Student()** is automatically called.

Types of Constructors in C++

Mainly used constructors are:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

Parameterized Constructor

A **parameterized constructor** is a constructor that **accepts parameters to initialize objects with different values**.

Example

```
#include<iostream>
using namespace std;

class Student
{
public:
   int roll;

   Student(int r)
   {
      roll = r;
   }
};

int main()
{
   Student s1(101);

   cout<<s1.roll;
}
```

Output
101
Here the constructor initializes roll using the parameter.

Multiple Constructors in a Class

A class can contain **more than one constructor**.
This is possible because of **constructor overloading**.
Different constructors have **different parameter lists**.

Example

```
#include<iostream>
using namespace std;

class Demo
{
public:
   int a,b;

   Demo()
   {
```

```
    a=10;
    b=20;
  }

  Demo(int x,int y)
  {
    a=x;
    b=y;
  }

  void display()
  {
    cout<<a<<" "<<b<<endl;
  }
};

int main()
{
  Demo d1;
  Demo d2(5,7);

  d1.display();
  d2.display();
}
```

Dynamic Initialization of Objects

Dynamic initialization means **initializing objects at runtime using constructors**.
Values are provided **during program execution** rather than fixed values.

Example

```
#include<iostream>
using namespace std;

class Interest
{
public:
  int p,r,t;

  Interest(int x,int y,int z)
  {
    p=x;
    r=y;
    t=z;
  }

  void calculate()
  {
    cout<<"Interest = "<<(p*r*t)/100;
  }
};
```

```
int main()
{
   int p,r,t;

   cout<<"Enter principal, rate and time: ";
   cin>>p>>r>>t;

   Interest obj(p,r,t);

   obj.calculate();
}
```

Here values are **entered at runtime**, so objects are **dynamically initialized**.

Copy Constructor

A **copy constructor** is used to **initialize one object using another object of the same class**.

Syntax

```
ClassName(ClassName &object)
{
}
```

Example

```
#include<iostream>
using namespace std;

class Demo
{
public:
   int x;

   Demo(int a)
   {
      x=a;
   }

   Demo(Demo &d)
   {
      x=d.x;
   }
};

int main()
{
   Demo d1(10);
   Demo d2=d1;
   cout<<d2.x;
}
```

Output

10

Cloning a Game Character

Imagine you are playing a video game where you have a character:

- *Name: Warrior*
- *Level: 10*
- *Health: 100*
- *Weapons: Sword, Shield*

Now, the game introduces a feature:

"Clone Character"

How It Works in Real Life

When you choose to clone:

- *A new character is created*
- *It has the **same level, health, and weapons***
- *It behaves exactly like the original at the start*

No need to manually create and configure a new character

Relating to Copy Constructor

- *Original character → **existing object***
- *Cloned character → **new object***
- *Cloning process → **copy constructor***

Instead of:

- *Creating a new character*
- *Setting level, health, weapons again*

You simply say:

"Create a copy of this character"

The system:

- *Copies all attributes*
- *Creates a new independent character*

Here:

- d1 is the original object
- d2 is initialized using d1

When Copy Constructor is Called

Copy constructor is called when:

1. Object is **initialized from another object**

Demo d2 = d1;

2. Object is **passed as argument to a function**
3. Object is **returned from a function**

Destructor in C++

A **destructor** is a special member function that is **automatically called when an object is destroyed**.
It is used to **release memory or resources used by the object**.

Characteristics of Destructor

1. Same name as class
2. Preceded by ~
3. No parameters
4. No return type
5. Automatically called when object goes out of scope

Syntax

```
~ClassName()
{
}
```

Example

```
#include<iostream>
using namespace std;

class Demo
{
public:
   Demo()
   {
      cout<<"Constructor called"<<endl;
   }

   ~Demo()
   {
      cout<<"Destructor called"<<endl;
   }
};

int main()
{
```

Imagine you check into a hotel room.
During your stay, the room is assigned to you:

- *Bed*
- *Lights*
- *AC*
- *Towels*

You use all these resources while you are staying.

What Happens When You Leave?
When you check out:
The hotel staff:

- *Cleans the room*
- *Turns off lights and AC*
- *Removes used items*
- *Prepares the room for the next guest*

Relating to Destructor

- *Hotel room booking* → ***object creation***
- *Your stay* → ***object is in use***
- *Checkout* → ***object is destroyed***
- *Cleaning process* → ***destructor***

```
    Demo d1;
}
```

Output

Constructor called
Destructor called

Explanation:

- Constructor runs when object is created
- Destructor runs when object is destroyed

Difference Between Constructor and Destructor

Feature	Constructor	Destructor
Purpose	Initialize object	Destroy object
Name	Same as class	Same as class with ~
Parameters	Can have parameters	Cannot have parameters
Overloading	Possible	Not possible
Execution	Object creation	Object destruction

Types of Inheritance in C++

There are **five main types of inheritance**.

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Single Inheritance

Single inheritance means **one derived class inherits from one base class**.

Example

```
#include<iostream>
using namespace std;
class Animal
{
public:
    void eat()
    {
        cout<<"Animal eats food"<<endl;
    }
};
class Dog : public Animal
{
public:
    void bark()
    {
        cout<<"Dog barks"<<endl;
```

Online Subscription System

*There is a general **User**:*

- *Name*
- *Email*

*Then there is a **Premium User**:*

- *Gets all User features*
- *Plus: Ad-free content, HD streaming*
- *User → Parent*
- *Premium User → Child*

One parent → one child

```
    }
};
int main()
{
    Dog d;
    d.eat();
    d.bark();
}
```

Multiple Inheritance

Multiple inheritance means **a class inherits from more than one base class**.

```
        Base1      Base2
          \         /
           \       /
            Derived
```

Example

```
#include<iostream>
using namespace std;

class Father
{
public:
    void show1()
    {
        cout<<"Father class"<<endl;
    }
};
class Mother
{
public:
    void show2()
    {
        cout<<"Mother class"<<endl;
    }
};
class Child : public Father, public Mother
{
};

int main()
{
    Child c;
    c.show1();
    c.show2();
}
```

Smart Employee System

An employee can have multiple skills:

- ***Developer***
  - *Coding skills*
- ***Designer***
  - *UI/UX skills*

*Now a **Full Stack Creator**:*

- *Has coding skills*
- *Has design skills*

Developer + Designer → Parent classes

Full Stack Creator → Child
One child, multiple parents

"Multiple = one child, many parents"

Multilevel Inheritance

Multilevel inheritance means **a derived class becomes a base class for another class**.

A
|
B
|
C

Example

```
#include<iostream>
using namespace std;
class A
{
public:
   void showA()
   {
      cout<<"Class A"<<endl;
   }
};

class B : public A
{
public:
   void showB()
   {
      cout<<"Class B"<<endl;
   }
};

class C : public B
{
public:
   void showC()
   {
      cout<<"Class C"<<endl;
   }
};

int main()
{
   C obj;
   obj.showA();
   obj.showB();
   obj.showC();
}
```

Online Learning Platform

Scenario

- ***User***
  - *Basic account*
- ***Student (inherits User)***
  - *Can enroll in courses*
- ***Certified Student (inherits Student)***
  - *Gets certificate after completion*

User → Student → Certified Student

Chain structure

Hierarchical Inheritance

Hierarchical inheritance means **multiple derived classes inherit from the same base class**.

```
    A
   / \
  B   C
```

Example

```
#include<iostream>
using namespace std;

class Animal
{
public:
   void eat()
   {
      cout<<"Animal eats food"<<endl;
   }
};

class Dog : public Animal
{
public:
   void bark()
   {
      cout<<"Dog barks"<<endl;
   }
};

class Cat : public Animal
{
public:
   void meow()
   {
      cout<<"Cat meows"<<endl;
   }
};

int main()
{
   Dog d;
   Cat c;

   d.eat();
   d.bark();

   c.eat();
   c.meow();
}
```

Smart Streaming Platform

Content can be of different types:

• ***Content***
o Title
o Duration

Now different types of content:

• ***Movie***
o Director
o Release Year

• ***TV Show***
o Seasons
o Episodes

• ***Documentary***
o Topic
o Narrator

Content → Parent class

Movie, TV Show, Documentary → Child classes

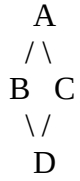
One parent, multiple children

"Hierarchical = one parent, many children"

Hybrid Inheritance

Hybrid inheritance is a **combination of two or more inheritance types**.
Example:

- Multiple + Multilevel
- Hierarchical + Multiple



This structure creates a **diamond problem**, which is solved using **virtual base class**.

Virtual Base Class

When multiple inheritance occurs, sometimes a **derived class receives multiple copies of a base class**, causing ambiguity.
To avoid this, C++ provides **virtual base class**.

Syntax

```
class B : virtual public A
{
};
```

Example

```
#include<iostream>
using namespace std;

class A
{
public:
   int x;
};
class B : virtual public A
{
};

class C : virtual public A
{
};

class D : public B, public C
{
};

int main()
{
   D obj;
   obj.x = 10;
```

```
   cout<<obj.x;
}
```

Here:

- Only **one copy of class A** exists.
- Ambiguity is removed.

Constructors in Derived Class

When an object of a **derived class is created**, constructors are called in this order:

1. Base class constructor
2. Derived class constructor

When the object is destroyed:

1. Derived class destructor
2. Base class destructor

Example

```
#include<iostream>
using namespace std;

class Base
{
public:
   Base()
   {
      cout<<"Base constructor"<<endl;
   }
};

class Derived : public Base
{
public:
   Derived()
   {
      cout<<"Derived constructor"<<endl;
   }
};

int main()
{
   Derived obj;
}
```

Output

```
Base constructor
Derived constructor
```

Parameterized Constructor in Inheritance

Sometimes the **base class constructor requires parameters**.
Then the derived class must pass values using **initializer list**.

Example

```
#include<iostream>
using namespace std;

class Base
{
public:
   int x;

   Base(int a)
   {
      x = a;
   }
};

class Derived : public Base
{
public:
   Derived(int a) : Base(a)
   {
      cout<<"Value = "<<x;
   }
};

int main()
{
   Derived obj(10);
}
```

Output

Value = 10

Polymorphism in C++

Polymorphism means **"one name, many forms."**
In object-oriented programming, polymorphism allows:

- A **single function name**
- A **single operator**

to perform **different operations depending on the situation**.
Example
A function add() can add:

- two numbers
- three numbers

Types of Polymorphism

Polymorphism is mainly classified into two types:

1. Compile-Time Polymorphism (Static Polymorphism)
2. Run-Time Polymorphism (Dynamic Polymorphism)

1. Compile-Time Polymorphism

This polymorphism is **resolved during compilation**.
The compiler determines **which function to call** before the program runs.
It is achieved using:

- **Function Overloading**
- **Operator Overloading**

Function Overloading

Function overloading means **multiple functions having the same name but different parameter lists**.
The compiler differentiates functions based on:

- number of arguments
- type of arguments
- order of arguments

Syntax

return_type function_name(parameters);

Multiple functions with the **same name but different parameters**.

Example

```
#include<iostream>
using namespace std;

class Demo
{
public:

   int add(int a,int b)
   {
      return a+b;
   }

   int add(int a,int b,int c)
   {
      return a+b+c;
   }
};

int main()
{
   Demo d;

   cout<<d.add(5,6)<<endl;
   cout<<d.add(2,3,4);
}
```

*When you say **"Order"** to a waiter, the basic action is the same — you want to place an order.*
*But the **details you provide change how that action is performed**.*

- *If you say "Order pizza" → the waiter brings a normal pizza*
- *If you say "Order pizza with extra cheese" → the waiter modifies the order*
- *If you say "Order pizza for 2 people" → the quantity or size changes*

*The **core action doesn't change**, but the **input details change the result**.*

Output
11
9
Explanation

- add(5,6) calls first function
- add(2,3,4) calls second function

Operator Overloading

Operator overloading allows **operators to work with user-defined objects**.
Normally operators work with **built-in data types**, but using operator overloading they can work with **class objects**.
Example operators: +, -, *,/ ==, <<, >>

Syntax
return_type operator operator_symbol(parameters)

Example

```
#include<iostream>
using namespace std;

class Number
{
public:
   int x;

   Number operator+(Number n)
   {
      Number temp;
      temp.x = x + n.x;
      return temp;
   }
};

int main()
{
   Number n1,n2,n3;

   n1.x = 10;
   n2.x = 20;

   n3 = n1 + n2;

   cout<<n3.x;
}
```

Output
30
Here the + operator is **overloaded** for objects.

*Imagine the **"+" operator means combining music** in a playlist.*

Now see how its meaning changes:

- *Song + Song*
  *→ You simply **add another song** to the playlist*
- *Playlist + Song*
  *→ You **insert a new song into an existing playlist***
- *Playlist + Playlist*
  *→ You **merge two playlists into one bigger playlist***
- *Song + Effect (like remix)*
  *→ Now "+" means **transforming the song** (adding beats, remixing)*

*The symbol **"+" is the same**, but:*

- *Sometimes it means **adding***
- *Sometimes it means **inserting***
- *Sometimes it means **merging***
- *Sometimes it means **modifying***

*The meaning changes based on **what is on the left and right side of "+"***

Friend Functions

A **friend function** is a function that is **not a member of the class but can access private and protected members of the class**.
Normally private members cannot be accessed outside the class, but **friend functions are allowed**.

Syntax

Inside class:
friend return_type function_name(parameters);

Example

```
#include<iostream>
using namespace std;

class Demo
{
private:
   int x;

public:
   Demo()
   {
      x=10;
   }

   friend void show(Demo);
};

void show(Demo d)
{
   cout<<"Value of x = "<<d.x;
}

int main()
{
   Demo obj;

   show(obj);
}
```

Imagine a bank system:

- *A **Bank Account** keeps details like balance, transactions, etc.*
- *These details are **private** — normal people cannot access them directly*

Now, there is a special person:
Auditor

- *The auditor is **not part of the bank account***
- *But still, they are allowed to **access private details** for checking*

Normally, only the bank account itself can access its private data

*But the auditor is given **special permission** to access it*

*This permission is **explicitly granted***

*This is exactly what a **friend function** is:*

- *It is **not a member of the class***
- *But it can **access private and protected data***
- *Because the class **trusts it and allows it***

Output

Value of x = 10

Explanation

- show() is **not a member function**
- But it can access private variable x.

Characteristics of Friend Functions

1. Declared using **friend keyword**
2. Defined **outside the class**
3. Not a member function
4. Can access **private and protected members**

Constant Functions

A **constant member function** is a function that **does not modify the data members of a class**.
To declare a constant function, the keyword **const** is used after the function declaration.

Syntax
return_type function_name() const;

Example

```
#include<iostream>
using namespace std;

class Demo
{
private:
   int x;

public:
   Demo()
   {
      x=10;
   }

   void display() const
   {
      cout<<"Value of x = "<<x;
   }
};

int main()
{
   Demo obj;
   obj.display();
}
```

Output
Value of x = 10

Imagine your TV remote:

- *There is a button that **shows current volume on screen***
- *When you press it:*
  - *It only **displays the volume level***
  - *It does **not increase or decrease the volume***
- *You are just **checking the current state***
- *No changes are made to the TV settings*
- *The system remains exactly the same*

This action is read-only

Rules of Constant Functions

1. Cannot modify data members
2. Cannot call non-constant functions
3. Used when function only **reads data**

Advantages of Constant Functions

- Protects object data from modification
- Improves program safety
- Useful in **large software systems**

Run Time Polymorphism in C++

Run-time polymorphism means **the function call is resolved at runtime rather than at compile time**.
This is also called:

- **Dynamic Binding**
- **Late Binding**

Run-time polymorphism is mainly achieved using:

- **Function Overriding**
- **Virtual Functions**

Function Overriding

Function overriding occurs when:

- A **base class** and **derived class** have a function with the **same name**
- Same **parameters**
- The **derived class redefines the function**

Example

```
#include<iostream>
using namespace std;

class Base
{
public:
   void show()
   {
      cout<<"Base class function"<<endl;
   }
};

class Derived : public Base
{
public:
   void show()
   {
      cout<<"Derived class function"<<endl;
   }
};

int main()
{
   Derived d;
   d.show();
}
```

Output

Derived class function

Navigation App Route Example (like Google Maps)
The function is: ***"Show Route(Source, Destination)"***

- *It always takes* ***same inputs*** *→ Source and Destination*

Now different behaviors:

- *For* ***car*** *→ shows highway-friendly routes*
- *For* ***bike*** *→ shows shorter/narrow roads*
- *For* ***walking*** *→ shows pedestrian paths*

In all cases:

- *You still give* ***Source and Destination***
- *You are not changing inputs*
- *Function name =* ***Show Route*** *(same)*
- *Parameters =* ***(Source, Destination)*** *(same)*
- *Implementation =* ***different in each case***

That's exactly what "same parameters" means in overriding

- *Same function name*
- *Same parameters*
- *But* ***child changes how it works***

Virtual Functions

A **virtual function** is a member function declared using the **virtual keyword** in the base class. It allows **dynamic binding**, meaning the function call is determined **at runtime** based on the object type.

Syntax

virtual return_type function_name();

Example

```
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show()
   {
      cout<<"Base class show"<<endl;
   }
};

class Derived : public Base
{
public:
   void show()
   {
      cout<<"Derived class show"<<endl;
   }
};

int main()
{
   Base* ptr;
   Derived obj;

   ptr = &obj;

   ptr->show();
}
```

Imagine a streaming app:

- *There is a common button: **"Play"***

Now different types of content:

- *If it's a **movie** → it plays like a full-length film*
- *If it's a **series episode** → it plays episode-wise*
- *If it's a **trailer** → it plays a short preview*

*The system only knows: **"Play"***

- *But the **actual behavior depends on the type of content***

At runtime, the app decides:

- *What exactly to play*
- *How to play it*

Output

Derived class show

Explanation:

- Pointer type → Base class
- Object type → Derived class
- Because of virtual, the **derived class function is executed**.

Pure Virtual Functions

A **pure virtual function** is a virtual function that **has no implementation in the base class**.
It forces the **derived class to implement the function**.

Syntax

```
virtual return_type function_name() = 0;
```

Example

```
#include<iostream>
using namespace std;

class Shape
{
public:
   virtual void draw() = 0;
};

class Circle : public Shape
{
public:
   void draw()
   {
      cout<<"Drawing Circle"<<endl;
   }
};
int main()
{
   Circle c;
   c.draw();
}
```

Here draw() is a **pure virtual function**.

Abstract Class

A class containing **at least one pure virtual function** is called an **abstract class**.
Characteristics:

- Cannot create objects of abstract class
- Used as **base class**
- Derived classes must **implement pure virtual functions**

Example

```
class Shape
{
public:
   virtual void draw() = 0;
};
```

Shape is an **abstract class**.

Interfaces in C++

C++ does not have a special interface keyword like some other languages.
Instead, **interfaces are implemented using abstract classes**.
An interface class:

- Contains **only pure virtual functions**
- No data members
- Used to define **common behavior**

Example

```
class Vehicle
{
public:
   virtual void start() = 0;
   virtual void stop() = 0;
};
```

Any class implementing this must define both functions.

Type Casting in C++

C++ provides **four type-casting operators**.
These are safer than traditional C-style casting.

1. static_cast

Used for **compile-time type conversion**.
Common uses:

- Convert numeric types
- Upcasting in inheritance

Example

```
int a = 10;
double b;

b = static_cast<double>(a);
```

2. dynamic_cast

Used for **runtime type checking in inheritance**.
Mostly used with **base class pointers**.
Works only with **polymorphic classes (classes having virtual functions)**.

Example

```
Base* ptr = new Derived;

Derived* d = dynamic_cast<Derived*>(ptr);
```

If conversion is invalid, it returns:

- NULL (for pointers)

3. const_cast
Used to **add or remove const qualifier**.
Example
```
const int x = 10;
int* ptr = const_cast<int*>(&x);
```

4. reinterpret_cast
Used for **low-level conversions** between unrelated pointer types.
This is the **most dangerous type cast** and should be used carefully.
Example
```
int x = 10;
char* ptr = reinterpret_cast<char*>(&x);
```

Comparison of Type Casts

Cast	Purpose
static_cast	Compile-time type conversion
dynamic_cast	Runtime checking in inheritance
const_cast	Add or remove const qualifier
reinterpret_cast	Low-level pointer conversion

Types of Errors in C++
In C++, errors are generally classified into **three main types**:
1. **Compile-Time Errors**
2. **Run-Time Errors**
3. **Logical Errors**

Exception handling mainly deals with **Run-Time Errors**.

1. Compile-Time Errors
Compile-time errors occur **during compilation of the program**.
The compiler detects these errors **before the program runs**.
- Missing semicolon
- Syntax mistakes
- Undeclared variables
- Type mismatch

Example
```
#include<iostream>
using namespace std;

int main()
{
   int a = 10
   cout<<a;
}
```
Error: **missing semicolon**
This error is detected by the **compiler**, so the program **will not run**.
Exception handling **cannot handle compile-time errors**.

2. Run-Time Errors

Run-time errors occur **while the program is executing**.
The program compiles successfully but **fails during execution**.
Examples:

- Division by zero
- File not found
- Invalid array index
- Memory allocation failure

Example

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10;
   int b = 0;

   cout<<a/b;
}
```

Here the program compiles successfully but crashes during execution.
Exception handling is used to handle these run-time errors.

Example Using Exception Handling

```
#include<iostream>
using namespace std;

int main()
{
   int a = 10;
   int b = 0;

   try
   {
      if(b==0)
         throw b;

      cout<<a/b;
   }

   catch(int x)
   {
      cout<<"Division by zero error";
   }
}
```

Output
Division by zero error
Instead of crashing, the program **handles the error properly**.

3. Logical Errors

Logical errors occur when the **program runs but produces incorrect output**.
The compiler **cannot detect these errors**.
Example

```
#include<iostream>
using namespace std;

int main()
{
   int a=10,b=5;

   cout<<"Sum = "<<a-b;
}
```

Output
Sum = 5
But the correct sum should be **15**.
This is a **logical error**.
Exception handling **cannot handle logical errors**.

Exception Handling in C++

Exception handling is a mechanism used to **handle runtime errors in a program**.
When an error occurs during program execution, the program normally **terminates abruptly**.
Using exception handling, we can **detect and handle errors properly** so the program continues running.
Examples of runtime errors:

- Division by zero
- File not found
- Invalid input
- Memory allocation failure

Exception handling helps make programs **more robust and reliable**.

Basic Concepts of Exception Handling

C++ exception handling is based on **three keywords**:

1. **try**
2. **throw**
3. **catch**

try Block
The try block contains **code that may generate an exception**.
Syntax

```
try
{
   // code that may cause exception
}
```

throw Keyword

The throw keyword is used to **generate an exception**.

Syntax

```
throw value;
```

Example

```
throw 10;
```

catch Block

The catch block is used to **handle the exception**.

Syntax

```
catch(data_type variable)
{
   // exception handling code
}
```

Example

```
#include<iostream>
using namespace std;

int main()
{
   int a,b;

   cout<<"Enter two numbers: ";
   cin>>a>>b;

   try
   {
      if(b==0)
      throw b;

      cout<<"Division = "<<a/b;
   }
   catch(int x)
   {
      cout<<"Division by zero not allowed";
   }

   return 0;
}
```

Output (if b = 0)

Division by zero not allowed

Explanation:

1. Code inside try is executed.
2. If b==0, an exception is thrown.
3. Control moves to the catch block.

Flow of Exception Handling

Program execution follows this order:

try → throw → catch

Steps:

1. Program executes code inside try
2. If an exception occurs → throw
3. Control transfers to matching catch block
4. Exception is handled

Multiple Catch Blocks

A program can have **multiple catch blocks** to handle different types of exceptions.

Example

```
#include<iostream>
using namespace std;

int main()
{
   try
   {
      throw 10;
   }

   catch(char c)
   {
      cout<<"Character exception";
   }

   catch(int x)
   {
      cout<<"Integer exception";
   }
}
```

Output
Integer exception
The matching catch block is executed.

Catch-All Handler

C++ allows a **catch-all handler** to catch any type of exception.

Syntax

```
catch(...)
{
   // handles all exceptions
}
```

Example

```
try
{
   throw "error";
}

catch(...)
{
   cout<<"Exception caught";
}
```

Rethrowing an Exception

Sometimes a catch block **cannot completely handle an exception**.
In that case, the exception can be **re-thrown**.

Example

```
#include<iostream>
using namespace std;

void test()
{
   try
   {
      throw 10;
   }

   catch(int x)
   {
      cout<<"Handled partially\n";
```

```
        throw;
    }
}

int main()
{
    try
    {
        test();
    }
    catch(int x)
    {
        cout<<"Handled in main";
    }
}
```

Specifying Exceptions

Older versions of C++ allowed functions to **specify which exceptions they may throw**.

Example

```
void test() throw(int,char)
{
}
```

This means the function may throw **int or char exceptions**.

However, **modern C++ discourages this syntax** and instead uses: noexcept

Example

```
void test() noexcept
{
}
```

This indicates the function **does not throw exceptions**.

Standard Exception Class

C++ provides standard exception classes in the **<exception>** header.

Example

```
#include<iostream>
#include<exception>
using namespace std;

int main()
{
    try
    {
        throw exception();
    }
    catch(exception& e)
    {
        cout<<"Standard exception caught";
    }
}
```

Advantages of Exception Handling

1. Separates **error handling code from normal code**
2. Makes programs **more reliable**
3. Allows **error propagation**
4. Improves **readability and maintainability**

Managing Console I/O Operations in C++

Console I/O operations are used to **take input from the keyboard and display output on the screen**.
In C++, input and output are performed using **streams**.
Example:
cin → input from keyboard
cout → output to screen
These operations are available through the **iostream library**.

C++ Streams

A **stream** is a flow of data between a program and an input/output device.
Types of streams:

1. **Input Stream**
2. **Output Stream**

Input Stream
Data flows **from input device to program**.
Example
```
cin >> x;
```
Here:
Keyboard → Program

Output Stream
Data flows **from program to output device**.
Example
```
cout << x;
```
Here:
Program → Screen

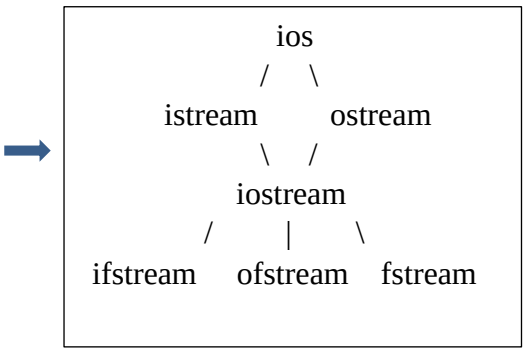
Common Stream Objects

Stream	Purpose
cin	Standard input stream
cout	Standard output stream
cerr	Standard error output
clog	Buffered error message

C++ Stream Classes

The C++ input-output system is based on a **hierarchy of classes**.
Main stream classes:

Class	Purpose
ios	Base class for all streams
istream	Input stream class
ostream	Output stream class
iostream	Input and output stream
ifstream	File input stream
ofstream	File output stream
fstream	File input/output stream



Unformatted I/O Operations

Unformatted I/O functions **read or write data without formatting**.
These functions work **character by character**.
Common functions:

Function	Purpose
get()	Read single character
getline()	Read line of text
put()	Write single character
read()	Read block of data
write()	Write block of data

Example: get() and put()

```
#include<iostream>
using namespace std;

int main()
{
   char ch;

   cout<<"Enter a character: ";
   cin.get(ch);

   cout<<"You entered: ";
   cout.put(ch);
}
```

Example: getline()

```
#include<iostream>
using namespace std;

int main()
{
   char name[50];

   cout<<"Enter your name: ";
   cin.getline(name,50);

   cout<<"Name: "<<name;
}
```

getline() is used to **read a full line including spaces**.

Formatted I/O Operations

Formatted I/O means **input/output with formatting**.
These operations use:
<< insertion operator
>> extraction operator

Example

```
#include<iostream>
using namespace std;

int main()
{
   int a;
   float b;

   cout<<"Enter integer and float: ";
   cin>>a>>b;

   cout<<"Integer = "<<a<<endl;
   cout<<"Float = "<<b;
}
```

Managing Output with Manipulators

Manipulators are used to **format the output**.
They are defined in the **iomanip library**.
Example
#include<iomanip>

Common Manipulators

Manipulator	Purpose
endl	New line
setw()	Set field width
setprecision()	Set decimal precision
setfill()	Fill characters
fixed	Fixed decimal format
left	Left alignment
right	Right alignment

Example: setw()

```
#include<iostream>
#include<iomanip>
using namespace std;

int main()
{
   cout<<setw(10)<<"Hello";
}
```

Output
Hello
setw(10) sets the **width to 10 characters**.

Example: setprecision()

```
#include<iostream>
#include<iomanip>
using namespace std;

int main()
{
   float x = 12.34567;

   cout<<setprecision(4)<<x;
}
```

Output
12.35

Example: setfill()

```
#include<iostream>
#include<iomanip>
using namespace std;

int main()
{
   cout<<setw(10)<<setfill('*')<<"Hi";
}
```

Output

********Hi

Example: fixed and setprecision

```
#include<iostream>
#include<iomanip>
using namespace std;

int main()
{
   float x = 12.34567;

   cout<<fixed<<setprecision(2)<<x;
}
```

Output

12.35

Definition of File

A **file** is a collection of data stored permanently on a secondary storage device such as a hard disk.

- Stores data **permanently (non-volatile storage)**
- Data remains even after program execution ends
- Used for **large data storage and retrieval**

Types of Files

1. Text Files

Store data in **human-readable form**

Example:

Hello
123
ABC

Extension: .txt, .cpp, .log

2. Binary Files

Store data in **binary format (0s and 1s)**

Not human-readable

Faster processing

Example:
Images, videos, compiled programs

File Operations
Basic operations on files:
1. Create a file
2. Open a file
3. Read from file
4. Write to file
5. Close file

File Handling in C++
C++ provides file handling using **stream classes** from:
#include <fstream>

What is a Stream?
A **stream** is a flow of data between:
- Program ↔ File

File Stream Classes
1. ifstream (Input File Stream)
- Used to **read data from file**

```
ifstream fin("file.txt");
```

2. ofstream (Output File Stream)
- Used to **write data to file**

```
ofstream fout("file.txt");
```

3. fstream (File Stream)
- Used for **both reading and writing**

```
fstream file;
```

Opening a File
Method 1: Using Constructor

```
ofstream fout("data.txt");
```

Method 2: Using open()

```
ofstream fout;
fout.open("data.txt");
```

File Open Modes:

Mode	Meaning
ios::in	Read mode
ios::out	Write mode
ios::app	Append mode
ios::binary	Binary mode

Mode	Meaning
ios::trunc	Delete old data
ios::ate	Start at end

Example
fstream file;
file.open("data.txt", ios::out | ios::app);
This opens file for **writing + appending**

Closing a File
file.close();
Why Important?
- Frees memory
- Saves data properly
- Avoids file corruption

Writing to a File
Using ofstream
ofstream fout("data.txt");
fout << "Hello World";
fout.close();

Writing Multiple Values
int a = 10;
float b = 5.5;

fout << a << " " << b;

Writing User Input to File
```
#include<iostream>
#include<fstream>
using namespace std;

int main()
{
   ofstream fout("data.txt");

   string name;
   cout<<"Enter name: ";
   cin>>name;

   fout<<name;

   fout.close();
}
```

Writing Character by Character

```
fout.put('A');
fout.put('B');
```

Reading from a File

Using ifstream

```
ifstream fin("data.txt");
string data;
fin >> data;
```

Reading Full Line

```
getline(fin, data);
```

Reading File Line by Line

```
while(getline(fin, data))
{
   cout << data << endl;
}
```

Reading Character by Character

```
char ch;
while(fin.get(ch))
{
   cout << ch;
}
```

Using fstream (Read + Write Together)

```
#include<iostream>
#include<fstream>
using namespace std;

int main()
{
   fstream file;

   // Writing
   file.open("data.txt", ios::out);
   file<<"C++ File Handling";
   file.close();

   // Reading
   file.open("data.txt", ios::in);
   string line;

   while(getline(file,line))
   {
```

```
        cout<<line;
    }

    file.close();
}
```

File Pointer Concepts

Each file has a **pointer position**:

- **get pointer (input pointer)** → reading
- **put pointer (output pointer)** → writing

Functions Related to Pointer

Function	Purpose
seekg()	Move read pointer
seekp()	Move write pointer
tellg()	Current read position
tellp()	Current write position

Example

```
file.seekg(0); // move to beginning
```

Checking File Status

Check if file opened successfully

```
ifstream fin("data.txt");

if(!fin)
{
    cout<<"File not found!";
}
```

End of File (EOF)

Using eof()

```
while(!fin.eof())
{
    getline(fin, data);
}
```

Better approach:

```
while(getline(fin, data))
```

Binary File Handling

Binary files store data in **raw format**.

Writing Binary File

```
#include<iostream>
#include<fstream>
using namespace std;
```

```
int main()
{
   ofstream file("data.dat", ios::binary);

   int num = 100;
   file.write((char*)&num, sizeof(num));

   file.close();
}
```

Reading Binary File

```
ifstream file("data.dat", ios::binary);

int num;
file.read((char*)&num, sizeof(num));
cout<<num;
```

Advantages of Binary Files

- Faster than text files
- No data conversion
- Efficient for objects

Difference: Text vs Binary File

Feature	Text File	Binary File
Format	Readable	Not readable
Speed	Slow	Fast
Storage	More space	Less space
Use	Simple data	Complex data

Important Functions Summary

Function	Use
open()	Open file
close()	Close file
getline()	Read line
get()	Read char
put()	Write char
write()	Write binary
read()	Read binary
eof()	End of file

Introduction to Templates

What are Templates?

A **template** in C++ is a feature that allows you to write **generic (general-purpose) code**. Instead of writing separate functions or classes for different data types, you can write **one template** that works for all types.

Why Templates are Needed

Without templates:

```
int add(int a, int b);
float add(float a, float b);
double add(double a, double b);
```

Same logic is repeated multiple times

With templates:

```
template <class T>
T add(T a, T b)
{
   return a + b;
}
```

One function works for all data types.

Key Advantages

- Code reusability
- Reduces duplication
- Type safety
- Easier maintenance

Syntax of Template

```
template <class T>
return_type function_name(parameters)
{
   // logic
}
```

OR

```
template <typename T>
```

class and typename are interchangeable in templates.

Function Templates

A **function template** allows a function to operate on **different data types**.

Example 1: Simple Function Template

```
#include<iostream>
using namespace std;

template <class T>
T add(T a, T b)
{
   return a + b;
}
int main()
{
```

```
    cout << add(10, 20) << endl;       // int
    cout << add(5.5, 3.2) << endl;    // float

    return 0;
}
```

How It Works

- The compiler generates separate functions:
  - add<int>()
  - add<float>()

This process is called **Template Instantiation**.

Example 2: Find Maximum

```
template <class T>
T maximum(T a, T b)
{
    return (a > b) ? a : b;
}
```

Function Template with Multiple Parameters

```
template <class T, class U>
void display(T a, U b)
{
    cout << a << " " << b;
}
```

Function Overloading vs Templates

Feature	Function Overloading	Templates
Code duplication	Yes	No
Flexibility	Less	More
Maintenance	Difficult	Easy

Class Templates

A **class template** allows a class to work with **different data types**.

Syntax

```
template <class T>
class ClassName
{
    T data;

public:
    void setData(T d)
    {
        data = d;
```

```
    }

    void display()
    {
        cout << data;
    }
};
```

Example: Class Template

```
#include<iostream>
using namespace std;

template <class T>
class Test
{
    T value;

public:
    Test(T v)
    {
        value = v;
    }

    void show()
    {
        cout << value << endl;
    }
};

int main()
{
    Test<int> obj1(10);
    Test<float> obj2(5.5);

    obj1.show();
    obj2.show();

    return 0;
}
```

How It Works

- The compiler creates:
    - Test<int>
    - Test<float>

Class Template with Multiple Types

```
template <class T1, class T2>
class Data
{
    T1 a;
    T2 b;
```

```
public:
   Data(T1 x, T2 y)
   {
      a = x;
      b = y;
   }

   void display()
   {
      cout << a << " " << b;
   }
};
```

Example
Data<int, float> obj(10, 5.5);

Key Concepts of Templates

Template Instantiation

- Compiler generates actual code when template is used

Generic Programming

- Writing programs independent of data types

Reusability

- Same code reused for different data types

Advantages of Templates

- Reduces code duplication
- Increases flexibility
- Improves readability
- Type-safe (checked at compile time)

Disadvantages of Templates

- Code complexity increases
- Compilation time may increase
- Error messages can be complex

Real-Life Use

Templates are widely used in:

- Standard Template Library (STL)
    - vector
    - list
    - map
    - stack

Introduction to C++ Standard Library

What is the C++ Standard Library?

The **C++ Standard Library (STL)** is a collection of **predefined classes and functions** that help programmers perform common tasks efficiently.
It provides ready-made components such as:

- Data structures
- Algorithms
- Iterators
- Input/Output tools

Components of STL

Containers

Used to store data.
Examples:

- vector
- list
- queue
- stack
- map

Algorithms

Used to perform operations like:

- Sorting
- Searching
- Reversing

Example:

```
sort(arr, arr+n);
```

Iterators

Used to traverse elements in containers.
Example:

```
vector<int>::iterator it;
```

Advantages of STL

- Saves development time
- Efficient and optimized
- Reduces code complexity
- Reusable components

Working with Stack, Vector, Queue, Map

Vector

A **vector** is a dynamic array that can grow or shrink in size.
Header File
#include<vector>

Example

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
   vector<int> v;

   v.push_back(10);
   v.push_back(20);
   v.push_back(30);

   for(int i=0; i<v.size(); i++)
   {
      cout<<v[i]<<" ";
   }
}
```

Common Functions

Function	Description
push_back()	Add element
pop_back()	Remove last element
size()	Number of elements
at()	Access element
clear()	Remove all elements

Stack

A **stack** is a linear data structure that follows:
LIFO (Last In First Out)

Header File
#include<stack>

Example

```
#include<iostream>
#include<stack>
using namespace std;
```

```
int main()
{
   stack<int> s;

   s.push(10);
   s.push(20);
   s.push(30);

   cout<<s.top()<<endl;

   s.pop();

   cout<<s.top();
}
```

Common Functions

Function	Description
push()	Insert element
pop()	Remove element
top()	Access top element
empty()	Check if empty

Queue

A **queue** follows:
FIFO (First In First Out)
Header File
#include<queue>

Example

```
#include<iostream>
#include<queue>
using namespace std;

int main()
{
   queue<int> q;
   q.push(10);
   q.push(20);
   q.push(30);

   cout<<q.front()<<endl;

   q.pop();

   cout<<q.front();
}
```

Common Functions

Function	Description
push()	Insert element
pop()	Remove element
front()	First element
back()	Last element

Map

A **map** stores data in **key-value pairs**.

- Keys are unique
- Automatically sorted

Header File

```
#include<map>
```

Example

```
#include<iostream>
#include<map>
using namespace std;

int main()
{
   map<int, string> m;

   m[1] = "A";
   m[2] = "B";
   m[3] = "C";

   for(auto x : m)
   {
      cout<<x.first<<" "<<x.second<<endl;
   }
}
```

Common Functions

Function	Description
insert()	Insert pair
erase()	Remove element
find()	Search key
size()	Number of elements

Introduction to RTTI (Run-Time Type Information)

What is RTTI?

RTTI stands for **Run-Time Type Information**.
It allows the program to determine the **type of an object at runtime**.

Why RTTI is Needed

- Used in **polymorphism**
- Helps identify object type during execution
- Useful in **dynamic casting**

Features of RTTI

- Uses typeid operator
- Uses dynamic_cast
- Works with **polymorphic classes (with virtual functions)**

---

typeid Operator

Syntax

typeid(variable).name()

Example

```
#include<iostream>
#include<typeinfo>
using namespace std;

int main()
{
   int a = 10;
   cout<<typeid(a).name();
}
```

dynamic_cast

Used to safely convert **base class pointer to derived class pointer**.

Example

```
#include<iostream>
using namespace std;

class Base
{
public:
   virtual void show() {}
};

class Derived : public Base
{
};
```

```
int main()
{
   Base* b = new Derived;

   Derived* d = dynamic_cast<Derived*>(b);

   if(d != NULL)
      cout<<"Successful casting";
   else
      cout<<"Failed casting";
}
```

Key Points about RTTI

- Works only with **virtual functions**
- Used for **safe type conversion**
- Helps avoid runtime errors

****
